

Wheelchair User Detection in Street Imagery

Transfer learning with YOLOv8 · Two hyperparameter search strategies

Emma Weinstein · Tal Matsil · Tamar Telele · github.com/Emma-V/cv-group-project

Applied Computer Vision · The Google & Reichman Tech School — the High-Tech School of Google and Reichman University

1 · Introduction and Motivation

Our goal is to give **accessibility consultants and urban planners** a practical AI tool: a detector that finds wheelchair users in street imagery, so that locations with relatively high wheelchair traffic can be identified and **prioritized for accessibility upgrades** — ramps, curb cuts, accessible crossings — instead of relying on expensive manual surveys. The task requires localisation, not only classification: knowing *where and how many* is what makes the output actionable.

The three-class taxonomy is deliberately hard: `people_wheelchair` (person + chair as one unit), `person` (pedestrian, no mobility aid) and `wheelchair` (unoccupied chair). The first two show the same body from the same angles, differing only by a chair that is often occluded by its occupant — separating them is exactly the capability the use case needs.

Rather than run one hyperparameter search, we ran **two independent searches following opposite philosophies** — a sequential one-variable-at-a-time search (Model A) and a broad Successive Halving search (Model B) — and treated the resulting models as candidates in a controlled selection by F1 on `people_wheelchair`.

2 · Dataset and Preprocessing

Source. A public Roboflow street-imagery dataset (YOLOv8 export, CC BY 4.0): 1,101 train / 236 validation / 236 test images, ~4,000 annotated objects across the three classes.

We did not use the dataset as originally published. The dataset was initially taken from Roboflow assuming it was in good condition, but issues surfaced gradually as work progressed. Cleaning happened in several rounds before a single training epoch was run — a model trained on dirty data learns the noise, not the signal.

Mislabelled boxes were identified and corrected by hand. **Background images** with no annotated objects were added so the model would learn that "nothing here" is a valid answer. An automated **near-duplicate check** found at least 10 image pairs that were nearly identical but for a small change — a shift, a slight zoom-in, or a colour shift — as if light augmentation had been applied to a subset of images without removing the originals; these were fixed and the check re-run to confirm the dataset was clean. **Object sizes** were measured in pixels against the COCO standard: only ~4% were found to be small, which justified using `imgsz=640` instead of a more expensive higher resolution. Finally, an additional QC pass ran the trained model across the entire dataset (not just the test split) to flag

images with repeated label-prediction mismatches as candidates for manual review in Roboflow.

2.1 Integrity validation

Check	Method	Result
Exact duplicates	MD5 hash	0 groups
Near-duplicates	Perceptual hash	~10 pairs found & fixed
Train ↔ test leak (exact)	MD5 cross-split	0 images
Train ↔ test leak (near)	Perceptual hash post-fix	0 near-dups
Label ambiguity	Cross-class IoU	2 / 1,606 > 0.5

Filename comparison is worthless here — Roboflow emits augmented copies under different names — so leakage is checked at byte level and perceptually. After fixing the near-duplicate pairs both returned zero, so the test score in §4 measures **generalisation, not memorisation**. The cross-class IoU check rules out annotation ambiguity in advance as an explanation for any weak class — a hypothesis we return to and reject in §4.4.

2.2 Class imbalance and remedy

Class	Objects	Share	Weight
<code>people_wheelchair</code>	843	21.2%	1.073
<code>person</code>	2,596	65.3%	0.332
<code>wheelchair</code>	539	13.5%	1.595

The imbalance is ~4.8:1, and the rare classes are the ones the project is about. Per-split shares track closely (person 62–66%, wheelchair 13–14%), so no split gives a distorted picture (Fig. 1). Rather than duplicating data, we corrected the imbalance **in the loss**: inverse-frequency weights normalised to mean 1.0. Ultralytics exposes a single scalar `cls`, so we passed the mean minority weight, **1.334** — derived from the data and applied identically to both models, so the fix cannot confound the comparison.

Augmentation follows the data. Both pipelines use flips, HSV colour shifts, scaling and mosaic (Model A adds mixup and copy-paste to boost rare-class exposure). EDA trimmed the search space before any GPU time was spent: image size stayed at 640 because only ~4% of boxes are small, and **vertical flipping was disabled** — an upside-down wheelchair never occurs in deployment, so training on one wastes capacity on a distribution that will never be evaluated. A single global seed (42) is fixed everywhere randomness enters the pipeline.

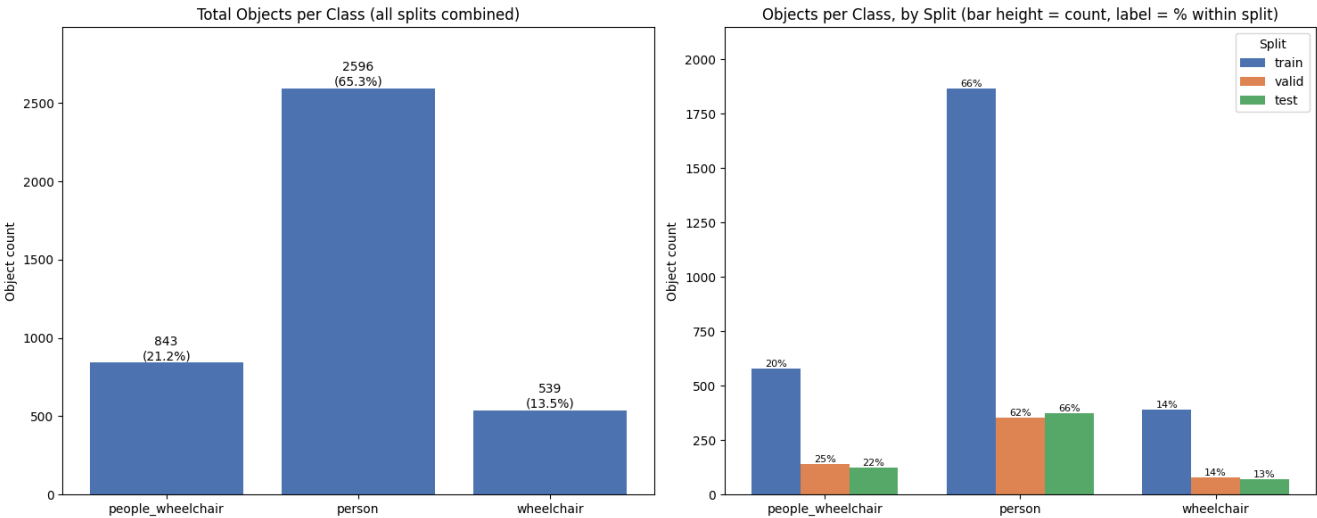


Fig. 1 — Class distribution. Left: total object counts — `person` dominates 65.3% vs 13.5% for `wheelchair`. Right: the same breakdown per split — proportions are consistent, so the imbalance, not a split skew, is the main finding.

3 · Model and Training — Two Search Philosophies

Backbone: YOLOv8 pretrained on COCO, fine-tuned on our three classes (transfer learning). Both searches used validation only; the final models share the same data, seed, class weight (1.334), image size and epoch budget — **only the search strategy differs, which is what we set out to study.**

3.1 Model A — sequential search, one axis at a time

One variable changes per run, so any difference has exactly one cause — the trajectory of experiments *is* the explanation of the final recipe:

#	Change (axis)	Outcome / decision
0	Base: yolov8s, 30 ep	Pipeline verified before long runs
1	copy_paste = 0.4	+ yolov8m; rare classes improve
2	copy_paste = 0.5	No gain → parameter saturated
3	cls = 0.8	Class errors in conf. matrix → helps
4	cls = 1.0	Worse → optimum bracketed at 0.8
5	freeze 10 → 0	Two-phase transfer; exposed a bug
6	SGD, lr0 = 0.001	First run where lr0 truly applied
7	lr0 = 0.0005 + cos_lr	Smoother late convergence
8	mixup, scale, HSV	Target the residual recall gap

Selection was not narrative: every checkpoint was re-evaluated on validation under identical inference settings and the winner picked automatically.

What the sequential discipline caught. Experiment 5's logs revealed that optimizer="auto" — the Ultralytics default — silently overrides any lr0 passed to it. Every earlier run had been sweeping a parameter that was never applied. A broad random search would have absorbed this as noise.

3.2 Model B — Successive Halving over 32 configs

Nine hyperparameters at ~30 min/run rule out exhaustive search; Successive Halving runs many random configs briefly and keeps the better half:

Stage	What happened	Outcome
A	32 configs → 16 → 8, 5 ep/rung	6 survivors
B1	Top-6 fully retrained, 40 ep	A27 wins (0.8809)
B2	A27 fixed; 5 architectures	yolov8s wins
C1	lr re-swept $\times 0.5 / \times 1 / \times 2$	$\times 0.5 \rightarrow 5.17\text{e-}05$
C2	freeze 10→0 vs single-stage	Single-stage wins
C3	Two seeds, final recipe	0.8899 ± 0.0017 — stable

3.3 Final recipes and experimental controls

	Model A	Model B
Backbone	YOLOv8m (25.8M)	YOLOv8s (11.1M)
Optimizer	SGD, lr0 = 0.001	Adam, lr0 = $5.17\text{e-}05$
Schedule	cos_lr, freeze = 10	cos_lr, freeze = 0
Augment.	strong (mixup, copy_paste)	mild (mosaic 0.5)
Epochs	80 (stopped 57)	80

Training behaviour (Figs. 2–3). Neither model shows classical overfitting: validation losses flatten rather than turning upward and mAP holds a plateau. What is present is a **train-validation gap** — train loss keeps falling after validation stalls, sharply so once mosaic switches off near epoch 70 (Model B): the models keep polishing their fit to the training images without getting better on new ones. That is saturation, not a U-shaped overfit curve, and A's early stop at epoch 57 (best weights: 27) was the right response. The honest claim is "no harmful overfitting on validation", not "perfect generalisation".

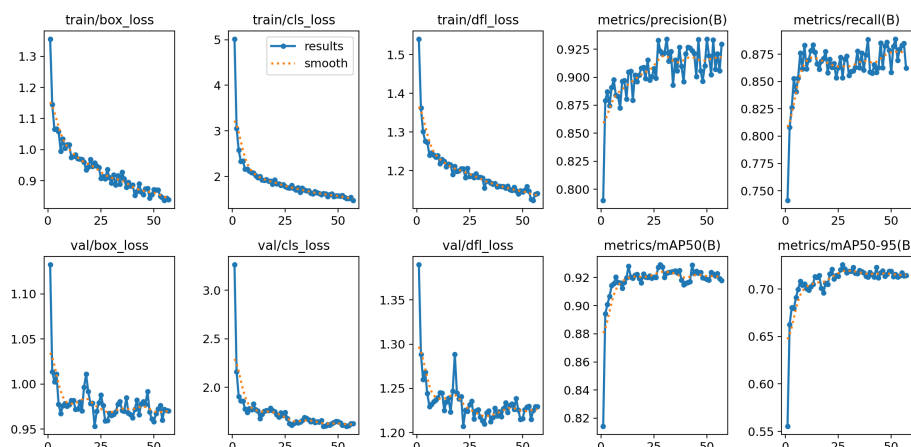


Fig. 2 — Model A training curves (YOLOv8m, SGD). Validation losses (bottom) flatten — no upward turn — and mAP plateaus. Best checkpoint epoch 27; early stop at 57.

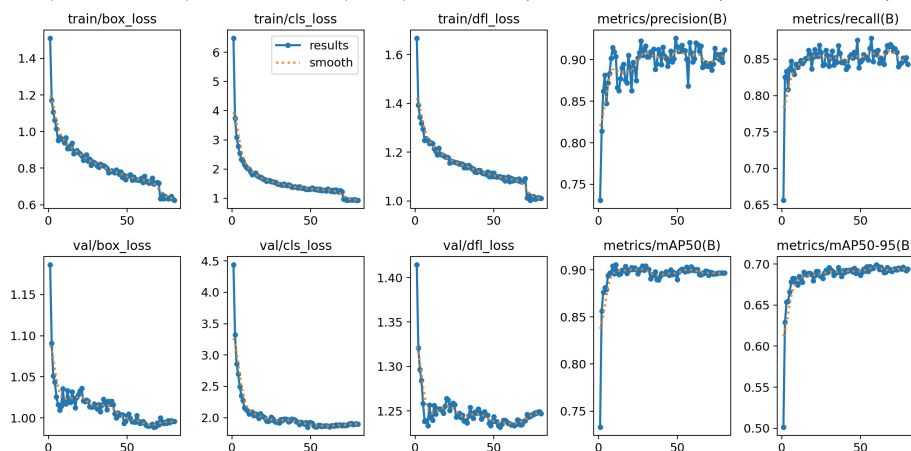


Fig. 3 — Model B training curves (YOLOv8s, Adam). The sharp train-loss drop at epoch ~70 (mosaic off) has no matching validation gain — fitting the train set, not learning.

4 · Results and Discussion

4.1 Overall validation — A vs B

Figs. 4–5 show the head-to-head. Aggregates favour A; the per-class view is more useful: on the two wheelchair classes the models are **effectively tied** (people_wheelchair mAP@0.5: 0.955 vs 0.945; wheelchair 0.990 vs 0.980). The entire gap lives in **person** (0.506 vs 0.445 at strict IoU) — "A is better at crowded pedestrians, and neither model has solved them".

4.2 Speed vs accuracy (Fig. 6)

No model dominates: A is more accurate but slower and noisier (38.9 ± 40.1 ms); B is faster and far steadier (30.2 ± 24.2 ms). **Conclusion: there is no single winner that is both faster and more precise** — pick A when detection quality matters more than milliseconds, B when throughput and latency stability matter and a modest mAP drop is acceptable.

4.3 Selection — F1 on people_wheelchair (Figs. 7–8)

Aggregate mAP is the wrong instrument under 4.8:1 imbalance — it is dominated by *person*. We selected on **F1 for people_wheelchair**, validation only. **Model B won — by 0.0012** (inside seed noise ± 0.0017): A via precision (0.974), B via recall (0.911) — and recall is the more valuable half here. B is also 2.3× smaller, 2.8× cheaper in compute and steadier in latency (Fig. 8).

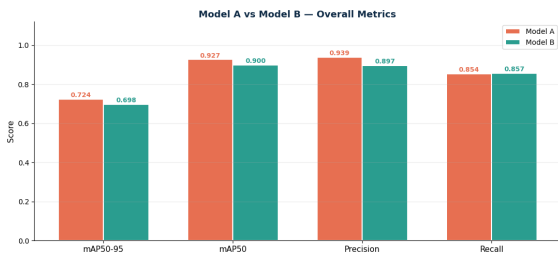


Fig. 4 — Overall validation metrics A vs B. A leads on mAP and precision; recall is nearly tied.

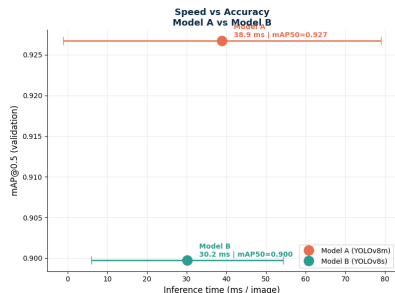


Fig. 6 — Speed vs accuracy. A: 38.9 ms, mAP50 = 0.927; B: 30.2 ms, mAP50 = 0.900. Neither sits at the ideal top-left — pick A for quality, B for throughput.

Model Efficiency — A vs B

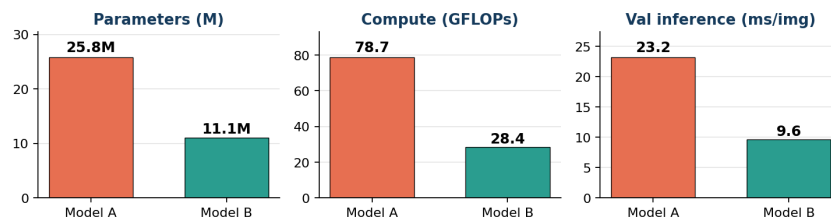


Fig. 8 — Model efficiency, A vs B. B is 2.3× smaller, 2.8× cheaper in compute, and 2.4× faster per image — the secondary criteria that break the F1 tie.

4.4 Threshold calibration & test

Confidence and NMS-IoU calibrated on validation (zero GPU cost): **conf = 0.45, IoU = 0.45**, both differing from COCO defaults. Test split touched only after every decision was locked.

Class	P	R	F1	mAP50	50-95
people_wheelchair	0.974	0.919	0.946	0.914	0.744
person	0.844	0.666	0.744	0.633	0.378
wheelchair	0.959	0.972	0.966	0.968	0.870
MACRO	0.926	0.853	0.885	0.839	0.664

wheelchair is close to solved (mAP@0.5 = 0.968); **people_wheelchair** holds F1 = 0.946 at recall 0.919 — about nineteen of every twenty users found. **person** is the weak point: precision (0.844) far exceeds recall (0.666) — missing a third of pedestrians.

4.5 Error analysis — the diagnosis (Figs. 9–10)

Of 625 test cases, 439 correct; errors: **missed 131 (70.4%)**, localization 36, background 12, classification 7, duplicate 0. **122 of 131 misses are person→background** — the whole recall gap in one number; class confusion is near-zero. Main driver is **crowding**: 54 count-mismatch images (worst: 32 labelled, 15 detected). Only 12 ghosts vs 131 misses — ample room to lower confidence cheaply.



Fig. 5 — Per-class breakdown. The gap is almost entirely in person at strict IoU (right); wheelchair classes are tied.

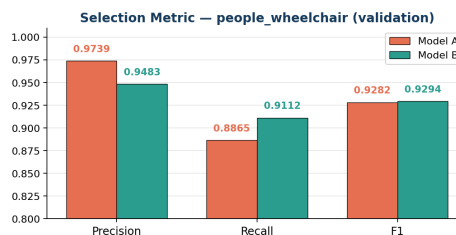


Fig. 7 — Selection metric, A vs B. Parity by different routes: A via precision, B via recall; B wins F1 by 0.0012 — inside seed noise.

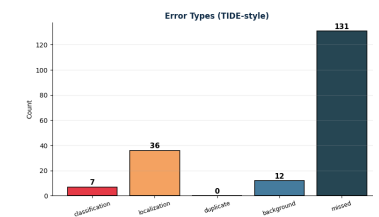


Fig. 9 — TIDE error categories on test. Missed detections dominate at 70.4%.



Fig. 10 — Misses by object size: 53.4% are medium/large — crowding, not size, is the real driver.

5 · Limitations, Conclusion & Future Work

Limitations. (1) Dense crowds are under-counted — the model finds the frame subject and drops the crowd behind it. (2) ~1,600 single-source images cannot cover every city, lighting or camera; night/rain/CCTV unverified. (3) Wheelchairs only — other mobility aids out of scope. (4) Thresholds tuned for F1; recall-first use needs re-calibration. (5) Not yet piloted with practitioners. (6) Selection margin is inside noise; *person* at 0.378 mAP@0.5:0.95 is not production-ready.

Conclusion. Target met: **F1 = 0.946 on people_wheelchair** and mAP@0.5 = 0.968 on wheelchair on unseen data (macro 0.839). Both searches converged within 0.0012 F1 — the ceiling is set by the data, not the search strategy.

Next steps: (1) class-specific confidence for *person* — 12 FPs vs 131 misses makes this the cheapest recall gain; (2) add dense-crowd data — failure concentrates in 54 images; (3) input resolution 960 / tiling for the 46.6% small-object misses; (4) review suspect labels — sets a realistic ceiling on recall; (5) ensemble A + B — complementary error profiles; (6) extend to video + tracking and pilot with planners — count over time.